

Mediation of Coping on Parents' Depression and Anxiety: Reproducible Report

Contents

1	Settings	2
1.1	Manage working environment	2
1.2	Required packages	2
1.3	Import and inspect data attributes	3
2	Data preprocessing	4
2.1	Standardize numeric variables	4
2.2	Check the correlation between numeric and ordinal background variables	4
2.3	Check the association between nominal background variables	5
2.4	Check the structure of the COPE-brief items	13
3	Perform bayesian structural equation modelling	15
3.1	Define functions	15
3.2	Call functions	27

1 Settings

1.1 Manage working environment

```
# Clear environment
rm(list=ls())

# Print the current R version
R.version.string

## [1] "R version 4.5.3 (2026-03-11 ucrt)"

# Increase the size of global objects that can be exported into 2GB
options(future.globals.maxSize = 2 * 1024^3)
```

1.2 Required packages

lavaan, blavaan, rstan : Perform Bayesian Structural Equation Modelling.

dplyr and tibble : Perform data manipulation and tabulation.

ggplot2 : Perform data visualization

```
packages <- c("lavaan", "blavaan", "rstan", "dplyr", "tibble", "ggplot2")

for (package in packages) {

  if (!requireNamespace(package, quietly = TRUE))
    install.packages(package)

  suppressPackageStartupMessages(library(package, character.only = TRUE))

}
```

1.3 Import and inspect data attributes

```
data <- read.csv("coping_data.csv")
attributes(data)[names(attributes(data)) != "row.names"]
```

```
## $names
## [1] "id"          "age"          "father"        "marital_stat"
## [5] "ses"         "residence"    "consanguinity" "living_sit"
## [9] "no_child"    "no_asd_child" "parent_cond"   "financial_s"
## [13] "c_age"       "time_since"   "delay"         "c_gender"
## [17] "birth_rank"  "comorbidity"  "phq9"          "gad7"
## [21] "sd"          "ac"           "de"            "es"
## [25] "is"          "bd"           "ve"            "pr"
## [29] "pl"          "hu"           "acc"           "re"
## [33] "sb"
##
## $class
## [1] "data.frame"
```

2 Data preprocessing

2.1 Standardize numeric variables

```
# Standardize continuous variables
cont_vars <- c("age", "ses", "c_age", "time_since", "delay",
              "sd", "ac", "de", "es", "is", "bd", "ve", "pr",
              "pl", "hu", "acc", "re", "sb", "phq9", "gad7")
data[cont_vars] <- scale(data[cont_vars])
```

2.2 Check the correlation between numeric and ordinal background variables

```
# Specify numeric background variables
num_vars <- c("age",
             "ses",
             "no_child",
             "no_asd_child",
             "c_age",
             "time_since",
             "delay"
            )

# Specify ordinal background variables
ord_vars <- c("birth_rank")

# Check the correlation across numeric and ordinal variables
vars_spearman <- c(num_vars, ord_vars)
cor_spearman <- cor(data[, vars_spearman],
                    method = "spearman",
                    use     = "pairwise.complete.obs"
                    )
as.data.frame(round(cor_spearman, 3))
```

```
##           age    ses no_child no_asd_child  c_age time_since  delay
## age          1.000  0.166   0.412      0.047  0.307      0.211  0.087
## ses          0.166  1.000  -0.220      0.171 -0.116      0.044 -0.042
## no_child     0.412 -0.220   1.000      0.004  0.332      0.242  0.002
## no_asd_child 0.047  0.171   0.004      1.000 -0.010      0.036  0.018
## c_age        0.307 -0.116   0.332     -0.010  1.000      0.677  0.261
## time_since   0.211  0.044   0.242      0.036  0.677      1.000 -0.132
## delay        0.087 -0.042   0.002      0.018  0.261     -0.132  1.000
## birth_rank   0.487 -0.138   0.783     -0.003  0.128      0.116 -0.033
##
##           birth_rank
## age              0.487
## ses             -0.138
## no_child         0.783
## no_asd_child    -0.003
## c_age            0.128
## time_since       0.116
## delay           -0.033
## birth_rank       1.000
```

2.3 Check the association between nominal background variables

```
# Specify nominal background variables
nom_vars <- c("father",
             "marital_stat",
             "residence",
             "consanguinity",
             "living_sit",
             "parent_cond",
             "c_gender",
             "comorbidity"
)

# Loop over nominal variables to check relations
results <- do.call(rbind, lapply(nom_vars, function(nom) {

  res_list <- list()

  # Nominal vs numeric variables
  for (v in num_vars) {

    # Perform Kruskal-Wallis test
    test <- kruskal.test(data[[v]] ~ data[[nom]])

    # Calculate effect size (eta-squared)
    H <- unname(test$statistic)
    k <- length(unique(data[[nom]]))
    n <- nrow(data)

    eta_sq <- (H - k + 1) / (n - k)
    eta_sq <- max(0, eta_sq)

    # Return a qualitative label for the effect size
    effect_label <- as.character(cut(eta_sq,
                                   breaks = c(-Inf, 0.01, 0.06, 0.14, Inf),
                                   labels = c("Negligible", "Small",
                                              "Moderate", "Large"),
                                   right = FALSE
                                   )
    )

  }

  # Append results
  res_list[[length(res_list) + 1]] <- data.frame(nominal_var = nom,
                                                  other_var = v,
                                                  test = "Kruskal-Wallis",
                                                  statistic = H,
                                                  p_value = test$p.value,
                                                  effect_size = eta_sq,
                                                  effect_label = effect_label,
                                                  stringsAsFactors = FALSE
                                                  )

})
```

```

# Nominal vs ordinal variables
for (v in ord_vars) {

  # Create a contingency table
  tab <- table(data[[nom]], data[[v]])
  tab <- tab[rowSums(tab) > 0, colSums(tab) > 0, drop = FALSE]

  # Perform Chi-square test
  test <- suppressWarnings(chisq.test(tab))

  # Calculate Cramer's V
  n <- sum(tab)
  chi_sq <- unname(test$statistic)
  V <- sqrt((chi_sq) / (n * min(nrow(tab) - 1, ncol(tab) - 1)))

  # Return a qualitative label for the effect size
  effect_label <- as.character(cut(V,
                                   breaks = c(-Inf, 0.10, 0.20,
                                              0.40, 0.60, 0.80, Inf),
                                   labels = c("Negligible", "Weak",
                                              "Moderate", "Relatively strong",
                                              "Strong", "Very strong"),
                                   right = FALSE
                                   )
  )

  # Append results
  res_list[[length(res_list) + 1]] <- data.frame(nominal_var = nom,
                                                  other_var = v,
                                                  test = "Chi-square",
                                                  statistic = chi_sq,
                                                  p_value = test$p.value,
                                                  effect_size = V,
                                                  effect_label = effect_label,
                                                  stringsAsFactors = FALSE
                                                  )
}

# Nominal vs nominal variables
for (v in setdiff(nom_vars, nom)) {

  # Create a contingency table
  tab <- table(data[[nom]], data[[v]])
  tab <- tab[rowSums(tab) > 0, colSums(tab) > 0, drop = FALSE]

  # Perform Chi-square test
  test <- suppressWarnings(chisq.test(tab))

  # Calculate Cramer's V
  n <- sum(tab)
  chi_sq <- unname(test$statistic)
  V <- sqrt(chi_sq / (n * min(nrow(tab) - 1, ncol(tab) - 1)))

```

```

# Return a qualitative label for the effect size
effect_label <- as.character(cut(V,
                                breaks = c(-Inf, 0.10, 0.20,
                                             0.40, 0.60, 0.80, Inf),
                                labels = c("Negligible", "Weak",
                                             "Moderate", "Relatively strong",
                                             "Strong", "Very strong"),
                                right = FALSE
                                )
)

# Append results
res_list[[length(res_list) + 1]] <- data.frame(nominal_var = nom,
                                                other_var = v,
                                                test = "Chi-square",
                                                statistic = chi_sq,
                                                p_value = test$p.value,
                                                effect_size = V,
                                                effect_label = effect_label,
                                                stringsAsFactors = FALSE
                                                )

}

do.call(rbind, res_list)
}))

# Calculated multiple-testing adjusted p-values
results$p_adj_bh <- p.adjust(results$p_value, method = "BH")

# Re-order columns
results <- results[, c("nominal_var", "other_var", "test", "statistic", "p_value",
                       "p_adj_bh", "effect_size", "effect_label")]

# Print results
results

```

	nominal_var	other_var	test	statistic	p_value
## 1	father	age	Kruskal-Wallis	1.313854e+01	2.892838e-04
## 2	father	ses	Kruskal-Wallis	5.038771e+00	2.478607e-02
## 3	father	no_child	Kruskal-Wallis	2.530859e+00	1.116394e-01
## 4	father	no_asd_child	Kruskal-Wallis	1.389905e+00	2.384211e-01
## 5	father	c_age	Kruskal-Wallis	9.298013e-01	3.349146e-01
## 6	father	time_since	Kruskal-Wallis	3.279606e-01	5.668624e-01
## 7	father	delay	Kruskal-Wallis	8.101645e-01	3.680716e-01
## 8	father	birth_rank	Chi-square	4.031130e+00	8.543039e-01
## 9	father	marital_stat	Chi-square	3.229662e-01	5.698305e-01
## 10	father	residence	Chi-square	1.525912e+01	9.372291e-05
## 11	father	consanguinity	Chi-square	3.258552e+00	7.105185e-02
## 12	father	living_sit	Chi-square	1.895548e+00	1.685775e-01
## 13	father	parent_cond	Chi-square	1.405574e+00	2.357926e-01
## 14	father	c_gender	Chi-square	1.777852e+00	1.824133e-01
## 15	father	comorbidity	Chi-square	1.972557e-30	1.000000e+00
## 16	marital_stat	age	Kruskal-Wallis	2.075577e+00	1.496733e-01
## 17	marital_stat	ses	Kruskal-Wallis	9.031689e-01	3.419334e-01

## 18	marital_stat	no_child	Kruskal-Wallis	4.200365e+00	4.041528e-02
## 19	marital_stat	no_asd_child	Kruskal-Wallis	1.560701e-01	6.928007e-01
## 20	marital_stat	c_age	Kruskal-Wallis	1.220256e+01	4.772398e-04
## 21	marital_stat	time_since	Kruskal-Wallis	6.414167e+00	1.132134e-02
## 22	marital_stat	delay	Kruskal-Wallis	3.291146e-01	5.661809e-01
## 23	marital_stat	birth_rank	Chi-square	5.784705e+00	6.713337e-01
## 24	marital_stat	father	Chi-square	3.229662e-01	5.698305e-01
## 25	marital_stat	residence	Chi-square	1.080214e-28	1.000000e+00
## 26	marital_stat	consanguinity	Chi-square	5.630880e-30	1.000000e+00
## 27	marital_stat	living_sit	Chi-square	8.828197e+00	2.966114e-03
## 28	marital_stat	parent_cond	Chi-square	1.127143e-01	7.370749e-01
## 29	marital_stat	c_gender	Chi-square	6.747561e-01	4.113983e-01
## 30	marital_stat	comorbidity	Chi-square	4.284434e+00	3.846289e-02
## 31	residence	age	Kruskal-Wallis	8.166502e-01	3.661614e-01
## 32	residence	ses	Kruskal-Wallis	1.250276e+00	2.634998e-01
## 33	residence	no_child	Kruskal-Wallis	1.578227e+00	2.090162e-01
## 34	residence	no_asd_child	Kruskal-Wallis	9.066148e-01	3.410142e-01
## 35	residence	c_age	Kruskal-Wallis	3.649435e-01	5.457735e-01
## 36	residence	time_since	Kruskal-Wallis	1.378262e-02	9.065436e-01
## 37	residence	delay	Kruskal-Wallis	1.454137e+00	2.278653e-01
## 38	residence	birth_rank	Chi-square	1.137846e+01	1.811631e-01
## 39	residence	father	Chi-square	1.525912e+01	9.372291e-05
## 40	residence	marital_stat	Chi-square	1.080214e-28	1.000000e+00
## 41	residence	consanguinity	Chi-square	4.241843e-02	8.368242e-01
## 42	residence	living_sit	Chi-square	2.560590e-01	6.128407e-01
## 43	residence	parent_cond	Chi-square	5.784809e-01	4.469084e-01
## 44	residence	c_gender	Chi-square	1.083885e+00	2.978301e-01
## 45	residence	comorbidity	Chi-square	2.819682e-01	5.954141e-01
## 46	consanguinity	age	Kruskal-Wallis	1.259741e+00	2.617001e-01
## 47	consanguinity	ses	Kruskal-Wallis	4.034480e+01	2.128711e-10
## 48	consanguinity	no_child	Kruskal-Wallis	9.449622e+00	2.111926e-03
## 49	consanguinity	no_asd_child	Kruskal-Wallis	1.644240e+00	1.997447e-01
## 50	consanguinity	c_age	Kruskal-Wallis	1.357396e+01	2.293446e-04
## 51	consanguinity	time_since	Kruskal-Wallis	4.922775e+00	2.650492e-02
## 52	consanguinity	delay	Kruskal-Wallis	2.472615e-02	8.750514e-01
## 53	consanguinity	birth_rank	Chi-square	1.334179e+01	1.006182e-01
## 54	consanguinity	father	Chi-square	3.258552e+00	7.105185e-02
## 55	consanguinity	marital_stat	Chi-square	5.630880e-30	1.000000e+00
## 56	consanguinity	residence	Chi-square	4.241843e-02	8.368242e-01
## 57	consanguinity	living_sit	Chi-square	1.280108e-01	7.205035e-01
## 58	consanguinity	parent_cond	Chi-square	6.161375e-02	8.039633e-01
## 59	consanguinity	c_gender	Chi-square	4.942218e-30	1.000000e+00
## 60	consanguinity	comorbidity	Chi-square	3.423546e+00	6.427282e-02
## 61	living_sit	age	Kruskal-Wallis	2.301052e+01	1.611177e-06
## 62	living_sit	ses	Kruskal-Wallis	4.370492e+00	3.656643e-02
## 63	living_sit	no_child	Kruskal-Wallis	5.912035e+00	1.503779e-02
## 64	living_sit	no_asd_child	Kruskal-Wallis	1.450071e-01	7.033532e-01
## 65	living_sit	c_age	Kruskal-Wallis	6.270072e-03	9.368864e-01
## 66	living_sit	time_since	Kruskal-Wallis	4.430096e-02	8.332947e-01
## 67	living_sit	delay	Kruskal-Wallis	2.195496e+00	1.384147e-01
## 68	living_sit	birth_rank	Chi-square	1.478166e+01	6.353259e-02
## 69	living_sit	father	Chi-square	1.895548e+00	1.685775e-01
## 70	living_sit	marital_stat	Chi-square	8.828197e+00	2.966114e-03
## 71	living_sit	residence	Chi-square	2.560590e-01	6.128407e-01

## 72	living_sit	consanguinity	Chi-square	1.280108e-01	7.205035e-01
## 73	living_sit	parent_cond	Chi-square	1.158336e+01	6.654446e-04
## 74	living_sit	c_gender	Chi-square	0.000000e+00	1.000000e+00
## 75	living_sit	comorbidity	Chi-square	4.211744e-02	8.373961e-01
## 76	parent_cond	age	Kruskal-Wallis	3.158034e+01	1.913573e-08
## 77	parent_cond	ses	Kruskal-Wallis	9.382412e-01	3.327306e-01
## 78	parent_cond	no_child	Kruskal-Wallis	7.090497e+00	7.749377e-03
## 79	parent_cond	no_asd_child	Kruskal-Wallis	1.170414e-02	9.138484e-01
## 80	parent_cond	c_age	Kruskal-Wallis	1.239116e-01	7.248301e-01
## 81	parent_cond	time_since	Kruskal-Wallis	6.038572e-01	4.371101e-01
## 82	parent_cond	delay	Kruskal-Wallis	6.420378e+00	1.128181e-02
## 83	parent_cond	birth_rank	Chi-square	2.481747e+01	1.669199e-03
## 84	parent_cond	father	Chi-square	1.405574e+00	2.357926e-01
## 85	parent_cond	marital_stat	Chi-square	1.127143e-01	7.370749e-01
## 86	parent_cond	residence	Chi-square	5.784809e-01	4.469084e-01
## 87	parent_cond	consanguinity	Chi-square	6.161375e-02	8.039633e-01
## 88	parent_cond	living_sit	Chi-square	1.158336e+01	6.654446e-04
## 89	parent_cond	c_gender	Chi-square	9.400246e-01	3.322715e-01
## 90	parent_cond	comorbidity	Chi-square	2.223175e-01	6.372798e-01
## 91	c_gender	age	Kruskal-Wallis	4.507114e-02	8.318733e-01
## 92	c_gender	ses	Kruskal-Wallis	4.144351e-01	5.197270e-01
## 93	c_gender	no_child	Kruskal-Wallis	7.467877e-04	9.781986e-01
## 94	c_gender	no_asd_child	Kruskal-Wallis	3.858523e-03	9.504697e-01
## 95	c_gender	c_age	Kruskal-Wallis	7.117058e-01	3.988781e-01
## 96	c_gender	time_since	Kruskal-Wallis	2.566642e-01	6.124212e-01
## 97	c_gender	delay	Kruskal-Wallis	1.171352e+00	2.791236e-01
## 98	c_gender	birth_rank	Chi-square	1.723084e+00	9.883566e-01
## 99	c_gender	father	Chi-square	1.777852e+00	1.824133e-01
## 100	c_gender	marital_stat	Chi-square	6.747561e-01	4.113983e-01
## 101	c_gender	residence	Chi-square	1.083885e+00	2.978301e-01
## 102	c_gender	consanguinity	Chi-square	4.942218e-30	1.000000e+00
## 103	c_gender	living_sit	Chi-square	0.000000e+00	1.000000e+00
## 104	c_gender	parent_cond	Chi-square	9.400246e-01	3.322715e-01
## 105	c_gender	comorbidity	Chi-square	3.854477e+00	4.961339e-02
## 106	comorbidity	age	Kruskal-Wallis	1.686534e+00	1.940585e-01
## 107	comorbidity	ses	Kruskal-Wallis	4.574079e+00	3.245919e-02
## 108	comorbidity	no_child	Kruskal-Wallis	2.753971e-01	5.997341e-01
## 109	comorbidity	no_asd_child	Kruskal-Wallis	5.448886e-01	4.604138e-01
## 110	comorbidity	c_age	Kruskal-Wallis	1.244169e+01	4.198552e-04
## 111	comorbidity	time_since	Kruskal-Wallis	2.320414e+00	1.276861e-01
## 112	comorbidity	delay	Kruskal-Wallis	6.077744e+00	1.368955e-02
## 113	comorbidity	birth_rank	Chi-square	1.390188e+01	8.435908e-02
## 114	comorbidity	father	Chi-square	1.972557e-30	1.000000e+00
## 115	comorbidity	marital_stat	Chi-square	4.284434e+00	3.846289e-02
## 116	comorbidity	residence	Chi-square	2.819682e-01	5.954141e-01
## 117	comorbidity	consanguinity	Chi-square	3.423546e+00	6.427282e-02
## 118	comorbidity	living_sit	Chi-square	4.211744e-02	8.373961e-01
## 119	comorbidity	parent_cond	Chi-square	2.223175e-01	6.372798e-01
## 120	comorbidity	c_gender	Chi-square	3.854477e+00	4.961339e-02
##	p_adj_bh	effect_size	effect_label		
## 1	4.959152e-03	4.087049e-02	Small		
## 2	1.416347e-01	1.359855e-02	Small		
## 3	3.620737e-01	5.154409e-03	Negligible		
## 4	5.502025e-01	1.312813e-03	Negligible		

## 5	6.513018e-01	0.000000e+00	Negligible
## 6	8.754867e-01	0.000000e+00	Negligible
## 7	6.795168e-01	0.000000e+00	Negligible
## 8	9.953055e-01	1.161122e-01	Weak
## 9	8.754867e-01	3.286570e-02	Negligible
## 10	2.249350e-03	2.259067e-01	Moderate
## 11	2.507712e-01	1.043943e-01	Weak
## 12	4.816499e-01	7.962177e-02	Negligible
## 13	5.502025e-01	6.856322e-02	Negligible
## 14	4.864354e-01	7.711027e-02	Negligible
## 15	1.000000e+00	8.122303e-17	Negligible
## 16	4.490198e-01	3.621473e-03	Negligible
## 17	6.513018e-01	0.000000e+00	Negligible
## 18	1.796235e-01	1.077564e-02	Small
## 19	9.409467e-01	0.000000e+00	Negligible
## 20	6.363197e-03	3.771906e-02	Small
## 21	7.547561e-02	1.822952e-02	Small
## 22	8.754867e-01	0.000000e+00	Negligible
## 23	9.259775e-01	1.390929e-01	Weak
## 24	8.754867e-01	3.286570e-02	Negligible
## 25	1.000000e+00	6.010620e-16	Negligible
## 26	1.000000e+00	1.372311e-16	Negligible
## 27	2.372891e-02	1.718306e-01	Weak
## 28	9.409467e-01	1.941574e-02	Negligible
## 29	7.259970e-01	4.750483e-02	Negligible
## 30	1.775211e-01	1.197047e-01	Weak
## 31	6.795168e-01	0.000000e+00	Negligible
## 32	5.855552e-01	8.426792e-04	Negligible
## 33	5.225405e-01	1.946894e-03	Negligible
## 34	6.513018e-01	0.000000e+00	Negligible
## 35	8.754867e-01	0.000000e+00	Negligible
## 36	1.000000e+00	0.000000e+00	Negligible
## 37	5.502025e-01	1.529081e-03	Negligible
## 38	4.864354e-01	1.950770e-01	Weak
## 39	2.249350e-03	2.259067e-01	Moderate
## 40	1.000000e+00	6.010620e-16	Negligible
## 41	9.851718e-01	1.191082e-02	Negligible
## 42	8.754867e-01	2.926405e-02	Negligible
## 43	7.553381e-01	4.398544e-02	Negligible
## 44	6.270108e-01	6.020824e-02	Negligible
## 45	8.754867e-01	3.070892e-02	Negligible
## 46	5.855552e-01	8.745490e-04	Negligible
## 47	2.554453e-08	1.324741e-01	Moderate
## 48	1.949470e-02	2.844991e-02	Small
## 49	5.099864e-01	2.169159e-03	Negligible
## 50	4.586892e-03	4.233658e-02	Small
## 51	1.445723e-01	1.320800e-02	Small
## 52	1.000000e+00	0.000000e+00	Negligible
## 53	3.353939e-01	2.112377e-01	Moderate
## 54	2.507712e-01	1.043943e-01	Weak
## 55	1.000000e+00	1.372311e-16	Negligible
## 56	9.851718e-01	1.191082e-02	Negligible
## 57	9.409467e-01	2.069130e-02	Negligible
## 58	9.851718e-01	1.435500e-02	Negligible

## 59	1.000000e+00	1.285658e-16	Negligible
## 60	2.410231e-01	1.070046e-01	Weak
## 61	6.444709e-05	7.410948e-02	Moderate
## 62	1.775211e-01	1.134846e-02	Small
## 63	9.022676e-02	1.653884e-02	Small
## 64	9.409467e-01	0.000000e+00	Negligible
## 65	1.000000e+00	0.000000e+00	Negligible
## 66	9.851718e-01	0.000000e+00	Negligible
## 67	4.258913e-01	4.025238e-03	Negligible
## 68	2.410231e-01	2.223443e-01	Moderate
## 69	4.816499e-01	7.962177e-02	Negligible
## 70	2.372891e-02	1.718306e-01	Weak
## 71	8.754867e-01	2.926405e-02	Negligible
## 72	9.409467e-01	2.069130e-02	Negligible
## 73	7.259396e-03	1.968257e-01	Weak
## 74	1.000000e+00	0.000000e+00	Negligible
## 75	9.851718e-01	1.186849e-02	Negligible
## 76	1.148144e-06	1.029641e-01	Moderate
## 77	6.513018e-01	0.000000e+00	Negligible
## 78	5.812033e-02	2.050672e-02	Small
## 79	1.000000e+00	0.000000e+00	Negligible
## 80	9.409467e-01	0.000000e+00	Negligible
## 81	7.553381e-01	0.000000e+00	Negligible
## 82	7.547561e-02	1.825043e-02	Small
## 83	1.669199e-02	2.880999e-01	Moderate
## 84	5.502025e-01	6.856322e-02	Negligible
## 85	9.409467e-01	1.941574e-02	Negligible
## 86	7.553381e-01	4.398544e-02	Negligible
## 87	9.851718e-01	1.435500e-02	Negligible
## 88	7.259396e-03	1.968257e-01	Weak
## 89	6.513018e-01	5.607045e-02	Negligible
## 90	8.892276e-01	2.726787e-02	Negligible
## 91	9.851718e-01	0.000000e+00	Negligible
## 92	8.543457e-01	0.000000e+00	Negligible
## 93	1.000000e+00	0.000000e+00	Negligible
## 94	1.000000e+00	0.000000e+00	Negligible
## 95	7.252329e-01	0.000000e+00	Negligible
## 96	8.754867e-01	0.000000e+00	Negligible
## 97	6.089969e-01	5.769432e-04	Negligible
## 98	1.000000e+00	7.591325e-02	Negligible
## 99	4.864354e-01	7.711027e-02	Negligible
## 100	7.259970e-01	4.750483e-02	Negligible
## 101	6.270108e-01	6.020824e-02	Negligible
## 102	1.000000e+00	1.285658e-16	Negligible
## 103	1.000000e+00	0.000000e+00	Negligible
## 104	6.513018e-01	5.607045e-02	Negligible
## 105	2.052968e-01	1.135395e-01	Weak
## 106	5.062397e-01	2.311561e-03	Negligible
## 107	1.693523e-01	1.203394e-02	Small
## 108	8.754867e-01	0.000000e+00	Negligible
## 109	7.673563e-01	0.000000e+00	Negligible
## 110	6.297828e-03	3.852422e-02	Small
## 111	4.032192e-01	4.445837e-03	Negligible
## 112	8.646029e-02	1.709678e-02	Small

## 113	2.892311e-01	2.156260e-01	Moderate
## 114	1.000000e+00	8.122303e-17	Negligible
## 115	1.775211e-01	1.197047e-01	Weak
## 116	8.754867e-01	3.070892e-02	Negligible
## 117	2.410231e-01	1.070046e-01	Weak
## 118	9.851718e-01	1.186849e-02	Negligible
## 119	8.892276e-01	2.726787e-02	Negligible
## 120	2.052968e-01	1.135395e-01	Weak

2.4 Check the structure of the COPE-brief items

```
# Specify the Brief-COPE model structure
cfa_model <- '
  # Problem-Focused Coping
  problem =~ ac + is + pl + pr

  # Emotion-Focused Coping
  emotion =~ ve + es + hu + acc + sb + re

  # Avoidant Coping
  avoid =~ sd + de + bd

  # Covariance
  problem ~~ emotion + avoid
  emotion ~~ avoid
'

fit_cfa <- cfa(cfa_model,
              data = data,
              estimator = "MLR"
            )
```

```
## Warning: lavaan->lav_lavaan_step11_estoptim():
##   the optimizer warns that a solution has NOT been found!
```

```
summary(fit_cfa, fit.measures = TRUE, standardized = TRUE)
```

```
## Warning: lavaan->lav_object_summary():
##   fit measures not available if model did not converge
```

```
## lavaan 0.6-21 did NOT end normally after 1229 iterations
## ** WARNING ** Estimates below are most likely unreliable
```

```
##
##      Estimator                      ML
##      Optimization method            NLMINB
##      Number of model parameters      29
##
##      Number of observations          299
##
## Parameter Estimates:
##
##      Standard errors                Sandwich
##      Information bread              Observed
##      Observed information based on    Hessian
##
## Latent Variables:
##      Estimate   Std.Err  z-value  P(>|z|)  Std.lv  Std.all
##      problem =~
##      ac          1.000           NA      0.603   0.604
##      is          1.137           NA      0.685   0.687
```

```

##      pl              1.034      NA              0.624      0.625
##      pr              1.379      NA              0.831      0.833
##      emotion =~
##      ve              1.000              0.000      0.000
##      es             53543.225      NA              0.678      0.678
##      hu             -7999.808      NA             -0.101     -0.102
##      acc            65820.135      NA              0.834      0.835
##      sb            -41256.261      NA             -0.523     -0.525
##      re             31719.629      NA              0.402      0.402
##      avoid =~
##      sd              1.000              0.000      0.000
##      de             62101.130      NA              0.561      0.562
##      bd             64393.480      NA              0.582      0.584
##
## Covariances:
##      Estimate      Std.Err  z-value  P(>|z|)  Std.lv  Std.all
##      problem ~~
##      emotion      0.000      NA          1.075    1.075
##      avoid      -0.000      NA         -1.051   -1.051
##      emotion ~~
##      avoid      -0.000      NA         -1.150   -1.150
##
## Variances:
##      Estimate      Std.Err  z-value  P(>|z|)  Std.lv  Std.all
##      .ac           0.633      NA          0.633    0.635
##      .is           0.527      NA          0.527    0.529
##      .pl           0.608      NA          0.608    0.610
##      .pr           0.306      NA          0.306    0.307
##      .ve           0.997      NA          0.997    1.000
##      .es           0.541      NA          0.541    0.541
##      .hu           0.986      NA          0.986    0.990
##      .acc          0.301      NA          0.301    0.302
##      .sb           0.718      NA          0.718    0.724
##      .re           0.836      NA          0.836    0.838
##      .sd           0.997      NA          0.997    1.000
##      .de           0.684      NA          0.684    0.685
##      .bd           0.656      NA          0.656    0.659
##      problem      0.364      NA          1.000    1.000
##      emotion      0.000      NA          1.000    1.000
##      avoid        0.000      NA          1.000    1.000

```

3 Perform bayesian structural equation modelling

3.1 Define functions

3.1.1 Build and fit Bayesian SEM models

```
# Define a function to build and fit Bayesian SEM models
fit_bsem <- function(model,
                      obs,
                      beta_sd,
                      burnin = 3000,
                      sample = 10000,
                      n.chains = 4,
                      target = "stan",
                      theta_prior = "cauchy(0, 1)[0, ]"){

  # Set a random seed for reproducibility
  set.seed(999)

  # Create the beta prior string
  beta_prior <- sprintf("normal(0, %.3f)", beta_sd)

  # Fit the model
  fit <- bsem(model,
             data = obs,
             burnin = burnin,
             sample = sample,
             n.chains = n.chains,
             dp = dpriors(beta = beta_prior,
                          theta = theta_prior
                          ),
             target = target
             )

  # Return the fitted model
  return(fit)
}
```

3.1.2 Calculate model diagnostics

```
# Define a function to report model diagnostics
diagnostics <- function(fit_bayes) {

  # Extract Stan objects
  sf <- fit_bayes@external$mcmcout
  sp <- get_sampler_params(sf, inc_warmup = FALSE)

  # Infer the no. of chains and iterations dynamically
  n_chains <- length(sp)
  n_iter <- nrow(sp[[1]])
  total_iter <- n_chains * n_iter

  # Divergent transitions
  divergences <- sapply(sp, function(x) sum(x[, "divergent__"]))
  total_div <- sum(divergences)

  # Treedepth hits
  max_td <- 10
  treedepth_hits <- sapply(sp, function(x) sum(x[, "treedepth__"] >= max_td))
  total_td <- sum(treedepth_hits)
  td_rate <- total_td / total_iter * 100

  # BFMI
  bfmi <- sapply(sp, function(x) {
    e <- x[, "energy__"]
    mean(diff(e)^2) / var(e)
  })

  min_bfmi <- min(bfmi)

  # R-hat
  rhats <- blavInspect(fit_bayes, "psrf")
  max_rhat <- max(rhats, na.rm = TRUE)

  # Effective sample size
  neff <- blavInspect(fit_bayes, "neff")
  min_ess <- min(neff, na.rm = TRUE)

  # Cutoffs
  cut_div <- 0
  cut_td_rate <- 0.1
  cut_bfmi <- 0.3
  cut_rhat <- 1.01
  cut_ess <- 400

  # Build diagnostic table
  diagnostics_df <- tibble(Diagnostic = c("Divergences", "Treedepth Hits",
                                           "Min BFMI", "Max R-hat", "Min ESS"),
                           Value = c(total_div, td_rate, min_bfmi,
                                       max_rhat, min_ess),
                           Cutoff = c(paste0("=", cut_div),
```

```

        paste0("< ", cut_td_rate, "%"),
        paste0("> ", cut_bfmi),
        paste0("< ", cut_rhat),
        paste0("> ", cut_ess)
    ),
    Convergence = c(
        ifelse(total_div == cut_div, "No Issue", "Issue"),
        ifelse(td_rate < cut_td_rate, "No Issue", "Issue"),
        ifelse(min_bfmi > cut_bfmi, "No Issue", "Issue"),
        ifelse(max_rhat < cut_rhat, "No Issue", "Issue"),
        ifelse(min_ess > cut_ess, "No Issue", "Issue")
    )
  ) %>%
  mutate(Value = case_when(Diagnostic == "Divergences" ~ as.character(Value),
    Diagnostic == "Treedepth Hits" ~ sprintf("%.3f%%", Value),
    Diagnostic == "Min BFMI" ~ sprintf("%.3f", Value),
    Diagnostic == "Max R-hat" ~ sprintf("%.5f", Value),
    Diagnostic == "Min ESS" ~ sprintf("%.0f", Value),
    TRUE ~ as.character(Value)
  )
)

# Print and return diagnostic table
print(diagnostics_df)
return(diagnostics_df)
}

```

3.1.3 Extract BSEM paths

```
# Define a function to extract Bayesian SEM paths
extract_paths <- function(fit_bayes, ci_probs = c(0.025, 0.975)) {

  # Extract Stan objects
  sf <- fit_bayes@external$mcmcout
  map <- fit_bayes@external$origpt

  # Create a data frame to store parameter mappings
  map_df <- data.frame(lhs = map$lhs,
                       op = map$op,
                       rhs = map$rhs,
                       pxname = map$pxnames,
                       stringsAsFactors = FALSE
  )

  # Keep regression paths only
  pe_reg <- subset(map_df, op == "~")

  # Posterior summaries
  stan_sum <- rstan::summary(sf, probs = ci_probs)$summary

  stan_df <- data.frame(pxname = rownames(stan_sum),
                        mean = stan_sum[, "mean"],
                        lower = stan_sum[, "2.5%"],
                        upper = stan_sum[, "97.5%"],
                        n_eff = stan_sum[, "n_eff"],
                        Rhat = stan_sum[, "Rhat"],
                        row.names = NULL,
                        stringsAsFactors = FALSE
  )

  # Merge mapping and summaries
  paths_df <- merge(pe_reg, stan_df, by = "pxname", all.x = TRUE)

  # Define a function to calculate the Probability of Direction (PD)
  pd_of <- function(param_name) {
    tryCatch({
      draws <- rstan::extract(sf, pars = param_name, permuted = FALSE)

      if (is.null(draws) || length(draws) == 0) {
        m <- rstan::as.matrix(sf, pars = param_name)
        if (is.null(m) || ncol(m) == 0) return(NA_real_)
        s <- m[, 1]
      } else {
        s <- as.vector(draws[, , 1])
      }

      max(mean(s > 0), mean(s < 0))
    }, error = function(e) NA_real_)
  }
}
```

```

# Calculate the probability of direction for all paths
paths_df$pd <- vapply(paths_df$pxname,
                      pd_of,
                      numeric(1)
                      )

# Final formatted data frame
paths <- paths_df %>%
  select(pxname,
         lhs,
         rhs,
         mean,
         pd,
         lower,
         upper,
         n_eff,
         Rhat
         ) %>%
  mutate(mean = round(mean, 4),
         pd   = round(pd, 4),
         lower = round(lower, 4),
         upper = round(upper, 4),
         n_eff = round(n_eff, 2),
         Rhat  = round(Rhat, 4)
         ) %>%
  mutate(row_id = row_number(), .before = 1)

return(paths)
}

```

3.1.4 Filter BSEM paths

```
# Define a function that filter paths
filter_paths <- function(paths_df,
  pd_threshold = 0.95,
  coping_vars = c("sd", "ac", "de", "es", "is", "bd", "ve",
    "pr", "pl", "hu", "acc", "re", "sb"),
  background_vars = c("age", "father", "ses", "residence",
    "consanguinity", "living_sit",
    "no_child", "parent_cond", "time_since",
    "delay", "c_gender", "comorbidity"),
  outcome_vars = c("phq9", "gad7")) {

  # Keep paths with probability of direction >= 0.95 and no 0 in 95% credible interval
  base_df <- paths_df %>%
    dplyr::filter(!is.na(pd), pd >= pd_threshold) %>%
    dplyr::filter(!is.na(lower), !is.na(upper), (lower > 0 | upper < 0))

  # Keep stable paths connecting background to outcomes
  df_bg_outcome <- base_df %>%
    dplyr::filter(lhs %in% outcome_vars, rhs %in% background_vars)

  # Keep stable paths connecting coping strategies to outcomes
  df_coping_outcome <- base_df %>%
    dplyr::filter(lhs %in% outcome_vars, rhs %in% coping_vars)

  # Store coping strategies connected to outcomes via stable paths
  retained_copings <- unique(df_coping_outcome$rhs)
  if (length(retained_copings) == 0) {
    stop("No coping strategies meet the Coping -> Outcome criteria.")
  }

  # Keep stable paths connecting background to selected coping strategies
  df_bg_coping <- base_df %>%
    dplyr::filter(lhs %in% retained_copings, rhs %in% background_vars)

  # Bind data frames and format
  filtered_paths_df <- dplyr::bind_rows(df_bg_outcome, df_coping_outcome, df_bg_coping) %>%
    dplyr::distinct(lhs, rhs, .keep_all = TRUE) %>%
    dplyr::mutate(from = rhs,
      to = lhs,
      effect = ifelse(mean > 0, "positive", "negative")
    ) %>%
    dplyr::select(pxname,
      from,
      to,
      mean,
      pd,
      lower,
      upper,
      n_eff,
      Rhat,
      effect
    )
}
```

```

    ) %>%
    dplyr::arrange(dplyr::desc(to),
                  dplyr::desc(abs(mean))
    )

    attr(filtered_paths_df, "retained_copings") <- retained_copings

    # Print and return filtered paths
    print(filtered_paths_df)
    return(filtered_paths_df)
}

```

3.1.5 Acquire draws from the posterior distributions

```
# Define a function to acquire posterior draws for retained paths
get_posterior <- function(fit,
                          retained_df,
                          model_name = NA_character_,
                          beta_sd    = NA_real_) {

  # Return an empty tibble if there are no retained paths
  if (is.null(retained_df) || nrow(retained_df) == 0) {
    return(dplyr::tibble())
  }

  # Convert MCMC output to matrix (rows = draws, cols = parameters)
  mat <- as.matrix(fit@external$mcmcout)

  # Subset draws to keep only retained paths
  px <- retained_df$pxname
  submat <- mat[, px, drop = FALSE]

  n_iter <- nrow(submat)
  n_par  <- ncol(submat)

  # Build long-format posterior draws table
  dplyr::tibble(model_name = model_name,
                beta_sd    = beta_sd,
                model      = paste0("beta_sd_", beta_sd),
                .iter      = rep(seq_len(n_iter), times = n_par),
                pxname     = rep(px, each = n_iter),
                draw       = as.vector(submat),
                from       = rep(retained_df$from, each = n_iter),
                to         = rep(retained_df$to, each = n_iter),
                mean       = rep(retained_df$mean, each = n_iter),
                lower      = rep(retained_df$lower, each = n_iter),
                upper      = rep(retained_df$upper, each = n_iter)
  )
}
```

3.1.6 Apply the BSEM pipeline

```
# Define a function to apply the Bayesian SEM pipeline
apply_pipeline <- function(model, data, beta_vec){

  # Create a list to store results
  results <- vector("list", length(beta_vec))
  names(results) <- paste0("beta_sd_", beta_vec)

  # Internal accumulators for retained paths and posterior draws
  kept_all <- dplyr::tibble()
  post_all <- dplyr::tibble()

  # Fit models, calculate diagnostics, extract paths, and get posteriors draws
  for (i in seq_along(beta_vec)) {

    beta <- beta_vec[i]
    nm   <- paste0("beta_sd_", beta)

    message("Performing BSEM with Beta prior SD = ", beta)

    fit   <- fit_bsem(model = model, obs = data, beta_sd = beta)
    diag  <- diagnostics(fit)
    paths <- extract_paths(fit)

    retained <- tryCatch(
      filter_paths(paths_df = paths),
      error = function(e) {
        message("No retained paths for beta_sd = ", beta, " (", e$message, ")")
        NULL
      }
    )

    # Append results
    results[[i]] <- list(beta_sd      = beta,
                        fit           = fit,
                        diagnostics    = diag,
                        paths_all      = paths,
                        paths_retained = retained
                      )

    if (!is.null(retained) && nrow(retained) > 0) {

      # Store all retained paths
      kept_all <- dplyr::bind_rows(kept_all,
                                   retained |>
                                   dplyr::transmute(model_name = nm,
                                                    beta_sd      = beta,
                                                    model         = nm,
                                                    from,
                                                    to,
                                                    pxname,
                                                    mean,
```

```

        lower,
        upper
      )
    )

    # Store all posterior draws
    post_all <- dplyr::bind_rows(post_all,
                                get_posterior(fit      = fit,
                                              retained_df = retained,
                                              model_name = nm,
                                              beta_sd   = beta
                                              )
                                )
  }
}

# Create a list to store the results of sensitivity analysis
sensitivity <- list(common_paths = NULL,
                    kw_table     = NULL,
                    large_paths  = NULL,
                    large_plot_data = NULL,
                    note         = NULL
                    )

# if no paths are retained in any model, stop sensitivity analysis
if (nrow(post_all) == 0) {
  sensitivity$note <- "No posterior draws available (no retained paths)."
  results$sensitivity <- sensitivity
  return(results)
}

# Create a unique identifier for each path (from -> to)
post_all %>%
  dplyr::mutate(key = paste(from, to, sep = "|")) -> post_all

# Split the path keys by model
keys_by_model <- split(post_all$key, post_all$model_name)

# Compute the intersection of path keys across all models
common_keys <- Reduce(intersect, keys_by_model)

# If no paths are common to all models, stop sensitivity analysis
if (length(common_keys) == 0) {
  sensitivity$note <- "Intersection empty: no path retained in all models."
  results$sensitivity <- sensitivity
  return(results)
}

# Keep only posterior draws for common paths
post_int <- post_all %>%
  dplyr::filter(key %in% common_keys)

```

```

# Store the list of common paths
sensitivity$common_paths <- post_int %>%
  dplyr::distinct(from, to)

# Compare between the draws of different models
kw_table <- post_int %>%
  dplyr::group_by(from, to) %>%
  dplyr::summarise(
    k = dplyr::n_distinct(model),
    n = dplyr::n(),
    kw = list(stats::kruskal.test(draw ~ model)),
    H = as.numeric(kw[[1]]$statistic),
    df = as.numeric(kw[[1]]$parameter),
    p_value = kw[[1]]$p.value,
    eta_sq = (H - k + 1) / (n - k),
    effect_label = as.character(cut(
      eta_sq,
      breaks = c(-Inf, 0.01, 0.06, 0.14, Inf),
      labels = c("Negligible", "Small", "Moderate", "Large"),
      right = FALSE
    )),
    .groups = "drop"
  ) %>%
  dplyr::select(-kw) %>% # Keep only scalar output
  dplyr::arrange(dplyr::desc(eta_sq)) # Rank paths by effect size

# Print and store results in the sensitivity analysis list
print(kw_table)
sensitivity$kw_table <- kw_table

# Retained common paths with large differences across beta SD
large_paths <- kw_table %>%
  dplyr::filter(effect_label %in% c("Large", "Moderate"))

sensitivity$large_paths <- large_paths

if (nrow(large_paths) > 0) {
  # Create keys for paths with large differences
  large_keys <- paste(large_paths$from, large_paths$to, sep = "|")

  # Retrieve the mean parameter and SD for parameters of paths with large differences
  large_summary <- post_int %>%
    dplyr::filter(key %in% large_keys) %>%
    dplyr::group_by(from, to, beta_sd) %>%
    dplyr::summarise(mean = mean(draw),
                     sd = sd(draw),
                     .groups = "drop")

  # Store the summary results
  sensitivity$large_summary <- large_summary

  # Plot the mean and SD for parameters of paths with large differences

```

```

sensitivity$large_plot <- ggplot(large_summary,
                                aes(x = beta_sd, y = mean, group = interaction(from, to))) +
  geom_line() +
  geom_point() +
  geom_errorbar(aes(ymin = mean - sd, ymax = mean + sd), width = 0.0) +
  facet_wrap(~ paste(from, "to", to), scales = "free_y") +
  labs(x = "Beta prior SD",
       y = "Posterior mean ( $\pm$  1 SD)") +
  theme_bw()

} else {
  sensitivity$large_summary <- NULL
  sensitivity$large_plot <- NULL
}

# Print the plot
print(sensitivity$large_plot)

# Append sensitivity analysis results
results$sensitivity <- sensitivity
return(results)
}

```

3.2 Call functions

```
# Specify the model structure
model <- '
  sd + ac + de + es + is + bd + ve + pr + pl + hu + acc + re + sb ~
    age + father + ses + residence + consanguinity + living_sit +
    no_child + parent_cond + time_since + delay + c_gender + comorbidity

  phq9 + gad7 ~ sd + ac + de + es + is + bd + ve + pr + pl + hu + acc + re + sb

  phq9 + gad7 ~
    age + father + ses + residence + consanguinity + living_sit +
    no_child + parent_cond + time_since + delay + c_gender + comorbidity

  phq9 ~~ gad7
'

# Define a beta prior SD vector
beta_vec = c(0.2, 0.30, 0.40, 0.5, 0.6, 0.7, 0.8, 0.9, 1)

# Apply the Bayesian SEM pipeline across different beta SD
res <- apply_pipeline(model = model, data = data, beta_vec = beta_vec)
```

```
## Performing BSEM with Beta prior SD = 0.2
```

```
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.009931 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 99.31 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration: 1300 / 13000 [10%] (Warmup)
## Chain 1: Iteration: 2600 / 13000 [20%] (Warmup)
## Chain 1: Iteration: 3001 / 13000 [23%] (Sampling)
## Chain 1: Iteration: 4300 / 13000 [33%] (Sampling)
## Chain 1: Iteration: 5600 / 13000 [43%] (Sampling)
## Chain 1: Iteration: 6900 / 13000 [53%] (Sampling)
## Chain 1: Iteration: 8200 / 13000 [63%] (Sampling)
## Chain 1: Iteration: 9500 / 13000 [73%] (Sampling)
## Chain 1: Iteration: 10800 / 13000 [83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 103.726 seconds (Warm-up)
## Chain 1:           264.496 seconds (Sampling)
## Chain 1:           368.222 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
```

```

## Chain 2:
## Chain 2: Gradient evaluation took 0.001762 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 17.62 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 93.74 seconds (Warm-up)
## Chain 2:           482.333 seconds (Sampling)
## Chain 2:           576.073 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001644 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 16.44 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 99.255 seconds (Warm-up)
## Chain 3:           266.232 seconds (Sampling)
## Chain 3:           365.487 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001597 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 15.97 seconds.
## Chain 4: Adjust your expectations accordingly!

```

```

## Chain 4:
## Chain 4:
## Chain 4: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 94.477 seconds (Warm-up)
## Chain 4:           263.838 seconds (Sampling)
## Chain 4:           358.315 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>  <chr>
## 1 Divergences    0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI      0.976    > 0.3  No Issue
## 4 Max R-hat     1.00017 < 1.01 No Issue
## 5 Min ESS       32160    > 400  No Issue
##           pxname      from to   mean   pd   lower  upper  n_eff
## 1  bet_sign[42]      father sd  0.3695 0.9986 0.1306 0.6077 51088.24
## 2  bet_sign[54]      father sb -0.2815 0.9912 -0.5111 -0.0503 51444.39
## 3  bet_sign[204] comorbidity sb  0.2606 0.9962 0.0699 0.4499 55176.15
## 4  bet_sign[189]    c_gender sb  0.2275 0.9821 0.0175 0.4369 57627.17
## 5  bet_sign[69]      ses     sb -0.1923 0.9992 -0.3077 -0.0765 46698.24
## 6  bet_sign[159]  time_since sb -0.1188 0.9832 -0.2276 -0.0087 50804.67
## 7  bet_sign[199] comorbidity pr -0.3078 0.9990 -0.5013 -0.1139 55896.76
## 8  bet_sign[184]    c_gender pr -0.2123 0.9754 -0.4243 -0.0008 58895.77
## 9  bet_sign[64]      ses     pr  0.1787 0.9986 0.0622 0.2963 45842.28
## 10 bet_sign[21]      acc phq9 -0.2550 1.0000 -0.3894 -0.1187 32159.51
## 11 bet_sign[25]      sb phq9  0.2297 1.0000 0.1366 0.3226 41447.89
## 12 bet_sign[55]      father phq9 -0.2135 0.9868 -0.4014 -0.0237 39451.16
## 13 bet_sign[15]      pr phq9 -0.1514 0.9929 -0.2720 -0.0307 40343.87
## 14 bet_sign[1]       sd phq9 -0.1477 1.0000 -0.2236 -0.0709 44731.29
## 15 bet_sign[205] comorbidity phq9  0.1473 0.9757 0.0010 0.2934 46278.38
## 16 bet_sign[160]  time_since phq9  0.1421 0.9997 0.0620 0.2227 40868.16
## 17 bet_sign[196] comorbidity is -0.2364 0.9907 -0.4314 -0.0396 55410.95
## 18 bet_sign[61]      ses     is  0.2257 0.9998 0.1084 0.3428 47586.00
## 19 bet_sign[166]    delay is  0.1086 0.9765 0.0018 0.2157 55136.06
## 20 bet_sign[22]      acc gad7 -0.3470 1.0000 -0.4806 -0.2119 32457.16
## 21 bet_sign[86]     residence gad7 -0.2324 0.9790 -0.4518 -0.0084 49036.83
## 22 bet_sign[206] comorbidity gad7  0.1713 0.9885 0.0245 0.3199 44685.71
## 23 bet_sign[16]      pr gad7 -0.1688 0.9966 -0.2896 -0.0480 37851.00
## 24 bet_sign[10]      is gad7 -0.1161 0.9781 -0.2285 -0.0033 34340.30
## 25 bet_sign[26]      sb gad7  0.1111 0.9902 0.0183 0.2035 42991.36

```

```

## 26 bet_sign[202] comorbidity acc -0.3106 0.9992 -0.5045 -0.1169 54537.54
## 27 bet_sign[187] c_gender acc -0.2450 0.9878 -0.4575 -0.0320 59198.15
## 28 bet_sign[52] father acc 0.2440 0.9798 0.0108 0.4754 48405.01
## 29 bet_sign[157] time_since acc 0.1323 0.9894 0.0205 0.2439 50248.26
## Rhat effect
## 1 1.0000 positive
## 2 0.9999 negative
## 3 0.9999 positive
## 4 0.9999 positive
## 5 0.9999 negative
## 6 1.0000 negative
## 7 0.9999 negative
## 8 1.0000 negative
## 9 0.9999 positive
## 10 1.0000 negative
## 11 1.0000 positive
## 12 1.0001 negative
## 13 1.0000 negative
## 14 1.0000 negative
## 15 1.0000 positive
## 16 1.0001 positive
## 17 1.0000 negative
## 18 1.0000 positive
## 19 1.0000 positive
## 20 1.0000 negative
## 21 1.0000 negative
## 22 1.0000 positive
## 23 1.0000 negative
## 24 1.0000 negative
## 25 1.0000 positive
## 26 1.0000 negative
## 27 1.0000 negative
## 28 0.9999 positive
## 29 1.0000 positive

```

```
## Performing BSEM with Beta prior SD = 0.3
```

```

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.002495 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 24.95 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration: 1300 / 13000 [10%] (Warmup)
## Chain 1: Iteration: 2600 / 13000 [20%] (Warmup)
## Chain 1: Iteration: 3001 / 13000 [23%] (Sampling)
## Chain 1: Iteration: 4300 / 13000 [33%] (Sampling)
## Chain 1: Iteration: 5600 / 13000 [43%] (Sampling)
## Chain 1: Iteration: 6900 / 13000 [53%] (Sampling)
## Chain 1: Iteration: 8200 / 13000 [63%] (Sampling)
## Chain 1: Iteration: 9500 / 13000 [73%] (Sampling)

```

```

## Chain 1: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 95.54 seconds (Warm-up)
## Chain 1: 392.459 seconds (Sampling)
## Chain 1: 487.999 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001896 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 18.96 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 99.123 seconds (Warm-up)
## Chain 2: 418.668 seconds (Sampling)
## Chain 2: 517.791 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001622 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 16.22 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:

```

```

## Chain 3: Elapsed Time: 96.369 seconds (Warm-up)
## Chain 3: 471.322 seconds (Sampling)
## Chain 3: 567.691 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001638 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 16.38 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 96.184 seconds (Warm-up)
## Chain 4: 265.482 seconds (Sampling)
## Chain 4: 361.666 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
## Diagnostic Value Cutoff Convergence
## <chr> <chr> <chr> <chr>
## 1 Divergences 0 = 0 No Issue
## 2 Treedepth Hits 0.000% < 0.1% No Issue
## 3 Min BFMI 0.978 > 0.3 No Issue
## 4 Max R-hat 1.00013 < 1.01 No Issue
## 5 Min ESS 32033 > 400 No Issue
## pxname from to mean pd lower upper n_eff
## 1 bet_sign[42] father sd 0.4708 0.9997 0.2006 0.7407 51375.63
## 2 bet_sign[54] father sb -0.3449 0.9959 -0.6008 -0.0873 46132.40
## 3 bet_sign[204] comorbidity sb 0.2965 0.9970 0.0881 0.5052 52666.77
## 4 bet_sign[189] c_gender sb 0.2628 0.9866 0.0315 0.4959 55761.81
## 5 bet_sign[69] ses sb -0.2020 0.9996 -0.3211 -0.0812 43238.21
## 6 bet_sign[159] time_since sb -0.1299 0.9880 -0.2430 -0.0176 51143.14
## 7 bet_sign[199] comorbidity pr -0.3517 0.9994 -0.5589 -0.1414 53526.32
## 8 bet_sign[184] c_gender pr -0.2457 0.9784 -0.4824 -0.0079 54191.20
## 9 bet_sign[64] ses pr 0.1858 0.9986 0.0642 0.3064 44483.22
## 10 bet_sign[21] acc phq9 -0.2694 0.9999 -0.4103 -0.1283 32033.25
## 11 bet_sign[55] father phq9 -0.2371 0.9881 -0.4448 -0.0323 39105.27
## 12 bet_sign[25] sb phq9 0.2324 1.0000 0.1371 0.3285 44637.84
## 13 bet_sign[145] parent_cond phq9 0.2062 0.9794 0.0089 0.4053 43614.74
## 14 bet_sign[205] comorbidity phq9 0.1663 0.9825 0.0122 0.3202 44720.97
## 15 bet_sign[15] pr phq9 -0.1489 0.9904 -0.2743 -0.0234 37693.45

```

```

## 16 bet_sign[1] sd phq9 -0.1482 0.9998 -0.2273 -0.0694 40731.90
## 17 bet_sign[160] time_since phq9 0.1476 0.9996 0.0652 0.2301 39519.29
## 18 bet_sign[196] comorbidity is -0.2705 0.9941 -0.4782 -0.0582 52826.05
## 19 bet_sign[61] ses is 0.2317 0.9999 0.1103 0.3540 47919.37
## 20 bet_sign[166] delay is 0.1153 0.9810 0.0055 0.2251 50685.84
## 21 bet_sign[22] acc gad7 -0.3664 1.0000 -0.5092 -0.2235 34633.93
## 22 bet_sign[86] residence gad7 -0.2777 0.9858 -0.5223 -0.0300 46525.18
## 23 bet_sign[206] comorbidity gad7 0.1914 0.9925 0.0372 0.3449 45706.69
## 24 bet_sign[16] pr gad7 -0.1654 0.9955 -0.2921 -0.0411 39433.21
## 25 bet_sign[10] is gad7 -0.1177 0.9761 -0.2345 -0.0011 36698.18
## 26 bet_sign[26] sb gad7 0.1103 0.9886 0.0152 0.2068 43781.21
## 27 bet_sign[202] comorbidity acc -0.3587 0.9995 -0.5693 -0.1500 50339.02
## 28 bet_sign[52] father acc 0.2956 0.9862 0.0345 0.5572 49300.31
## 29 bet_sign[187] c_gender acc -0.2824 0.9914 -0.5163 -0.0502 52690.25
## 30 bet_sign[157] time_since acc 0.1434 0.9931 0.0303 0.2562 49463.85
## Rhat effect
## 1 1.0000 positive
## 2 1.0000 negative
## 3 0.9999 positive
## 4 0.9999 positive
## 5 1.0000 negative
## 6 1.0000 negative
## 7 0.9999 negative
## 8 1.0000 negative
## 9 1.0000 positive
## 10 1.0000 negative
## 11 1.0000 negative
## 12 1.0000 positive
## 13 0.9999 positive
## 14 0.9999 positive
## 15 1.0000 negative
## 16 0.9999 negative
## 17 0.9999 positive
## 18 0.9999 negative
## 19 1.0001 positive
## 20 1.0000 positive
## 21 1.0001 negative
## 22 1.0000 negative
## 23 1.0000 positive
## 24 1.0000 negative
## 25 1.0000 negative
## 26 1.0001 positive
## 27 1.0000 negative
## 28 1.0000 positive
## 29 0.9999 negative
## 30 1.0000 positive

```

```
## Performing BSEM with Beta prior SD = 0.4
```

```
##
```

```
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
```

```
## Chain 1:
```

```
## Chain 1: Gradient evaluation took 0.001937 seconds
```

```
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 19.37 seconds.
```

```

## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 1: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 1: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 1: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 1: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 1: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 1: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 1: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 1: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 96.738 seconds (Warm-up)
## Chain 1:           264.522 seconds (Sampling)
## Chain 1:           361.26 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001841 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 18.41 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 96.646 seconds (Warm-up)
## Chain 2:           273.719 seconds (Sampling)
## Chain 2:           370.365 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001594 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 15.94 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:      1 / 13000 [ 0%] (Warmup)

```

```

## Chain 3: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 96.682 seconds (Warm-up)
## Chain 3: 261.161 seconds (Sampling)
## Chain 3: 357.843 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001639 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 16.39 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 161.113 seconds (Warm-up)
## Chain 4: 761.188 seconds (Sampling)
## Chain 4: 922.301 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>   <chr>
## 1 Divergences    0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI      0.971    > 0.3  No Issue
## 4 Max R-hat     1.00024  < 1.01 No Issue
## 5 Min ESS       28309    > 400  No Issue
##           pxname      from to mean   pd lower upper  n_eff
## 1 bet_sign[42] father sd 0.5246 0.9998 0.2373 0.8087 45507.01
## 2 bet_sign[54] father sb -0.3776 0.9961 -0.6520 -0.0990 44452.19
## 3 bet_sign[204] comorbidity sb 0.3121 0.9977 0.0979 0.5256 49647.97

```

```

## 4 bet_sign[189]      c_gender  sb  0.2779 0.9867  0.0317  0.5225 52337.59
## 5 bet_sign[69]       ses       sb -0.2053 0.9996 -0.3276 -0.0837 40177.49
## 6 bet_sign[159]     time_since sb -0.1347 0.9893 -0.2484 -0.0199 45542.45
## 7 bet_sign[199]     comorbidity pr -0.3711 0.9996 -0.5857 -0.1545 51232.33
## 8 bet_sign[184]     c_gender  pr -0.2588 0.9805 -0.5051 -0.0132 51653.41
## 9 bet_sign[64]      ses       pr  0.1890 0.9984  0.0659  0.3117 38998.49
## 10 bet_sign[21]     acc       phq9 -0.2749 0.9999 -0.4189 -0.1311 28960.42
## 11 bet_sign[55]     father    phq9 -0.2464 0.9889 -0.4601 -0.0341 31438.96
## 12 bet_sign[25]     sb       phq9  0.2338 1.0000  0.1383  0.3306 36439.26
## 13 bet_sign[145]    parent_cond phq9  0.2173 0.9804  0.0120  0.4236 37880.05
## 14 bet_sign[205]    comorbidity phq9  0.1748 0.9855  0.0178  0.3305 42839.60
## 15 bet_sign[100]    consanguinity phq9 -0.1726 0.9803 -0.3388 -0.0080 39278.13
## 16 bet_sign[160]    time_since phq9  0.1491 0.9998  0.0662  0.2327 35477.88
## 17 bet_sign[1]      sd       phq9 -0.1485 0.9999 -0.2267 -0.0694 37114.47
## 18 bet_sign[15]     pr       phq9 -0.1480 0.9890 -0.2717 -0.0218 34549.30
## 19 bet_sign[22]     acc       gad7 -0.3742 1.0000 -0.5199 -0.2273 28989.19
## 20 bet_sign[86]     residence gad7 -0.3004 0.9882 -0.5612 -0.0396 40854.83
## 21 bet_sign[206]    comorbidity gad7  0.2002 0.9933  0.0395  0.3578 43517.66
## 22 bet_sign[101]    consanguinity gad7 -0.1672 0.9758 -0.3331 -0.0011 39840.07
## 23 bet_sign[16]     pr       gad7 -0.1637 0.9940 -0.2914 -0.0367 34802.37
## 24 bet_sign[26]     sb       gad7  0.1105 0.9887  0.0150  0.2066 37362.55
## 25 bet_sign[202]    comorbidity acc -0.3778 0.9996 -0.5941 -0.1630 51026.56
## 26 bet_sign[52]     father    acc  0.3197 0.9883  0.0452  0.5968 43614.48
## 27 bet_sign[187]    c_gender  acc -0.2987 0.9908 -0.5425 -0.0518 53211.05
## 28 bet_sign[157]    time_since acc  0.1480 0.9942  0.0332  0.2646 46412.21
##      Rhat      effect
## 1  1.0000 positive
## 2  1.0000 negative
## 3  0.9999 positive
## 4  1.0000 positive
## 5  1.0000 negative
## 6  0.9999 negative
## 7  1.0000 negative
## 8  1.0000 negative
## 9  1.0000 positive
## 10 1.0001 negative
## 11 1.0000 negative
## 12 1.0000 positive
## 13 1.0000 positive
## 14 1.0000 positive
## 15 0.9999 negative
## 16 1.0001 positive
## 17 0.9999 negative
## 18 1.0001 negative
## 19 1.0001 negative
## 20 1.0000 negative
## 21 1.0000 positive
## 22 1.0000 negative
## 23 1.0000 negative
## 24 1.0000 positive
## 25 1.0000 negative
## 26 0.9999 positive
## 27 1.0000 negative
## 28 0.9999 positive

```

```

## Performing BSEM with Beta prior SD = 0.5

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001736 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 17.36 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [  0%] (Warmup)
## Chain 1: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 1: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 1: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 1: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 1: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 1: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 1: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 1: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 1: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 97.471 seconds (Warm-up)
## Chain 1:           285.214 seconds (Sampling)
## Chain 1:           382.685 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001876 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 18.76 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [  0%] (Warmup)
## Chain 2: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 98.205 seconds (Warm-up)
## Chain 2:           259.8 seconds (Sampling)
## Chain 2:           358.005 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).

```

```

## Chain 3:
## Chain 3: Gradient evaluation took 0.001796 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 17.96 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 155.862 seconds (Warm-up)
## Chain 3:                917.662 seconds (Sampling)
## Chain 3:                1073.52 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.011638 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 116.38 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 224.705 seconds (Warm-up)
## Chain 4:                721.434 seconds (Sampling)
## Chain 4:                946.139 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>   <chr>
## 1 Divergences      0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue

```

```

## 3 Min BFMI      0.951    > 0.3 No Issue
## 4 Max R-hat     1.00029 < 1.01 No Issue
## 5 Min ESS       26379    > 400 No Issue
##      pxname      from    to    mean    pd    lower    upper    n_eff
## 1  bet_sign[42]    father   sd  0.5524 0.9999 0.2575 0.8471 40113.87
## 2  bet_sign[54]    father   sb -0.3933 0.9972 -0.6725 -0.1134 42143.61
## 3  bet_sign[204]   comorbidity sb 0.3183 0.9976 0.0983 0.5362 44848.26
## 4  bet_sign[189]    c_gender sb 0.2862 0.9890 0.0412 0.5303 47276.54
## 5  bet_sign[69]      ses     sb -0.2069 0.9995 -0.3294 -0.0842 34425.45
## 6  bet_sign[159]   time_since sb -0.1370 0.9912 -0.2514 -0.0232 43403.78
## 7  bet_sign[199]   comorbidity pr -0.3819 0.9996 -0.6022 -0.1594 48288.27
## 8  bet_sign[184]    c_gender pr -0.2658 0.9818 -0.5153 -0.0153 48232.63
## 9  bet_sign[64]      ses     pr 0.1895 0.9986 0.0653 0.3131 38829.09
## 10 bet_sign[21]      acc    phq9 -0.2782 0.9998 -0.4254 -0.1314 26529.23
## 11 bet_sign[55]      father phq9 -0.2505 0.9875 -0.4666 -0.0322 30328.28
## 12 bet_sign[25]      sb     phq9 0.2343 1.0000 0.1385 0.3303 34660.46
## 13 bet_sign[145]   parent_cond phq9 0.2219 0.9816 0.0139 0.4310 36556.95
## 14 bet_sign[205]   comorbidity phq9 0.1791 0.9864 0.0209 0.3362 39650.76
## 15 bet_sign[100]  consanguinity phq9 -0.1779 0.9822 -0.3435 -0.0112 35469.72
## 16 bet_sign[160]   time_since phq9 0.1502 0.9998 0.0660 0.2341 34259.13
## 17 bet_sign[1]      sd     phq9 -0.1488 0.9999 -0.2282 -0.0696 35850.19
## 18 bet_sign[15]      pr     phq9 -0.1465 0.9876 -0.2744 -0.0178 31856.88
## 19 bet_sign[22]      acc    gad7 -0.3781 1.0000 -0.5250 -0.2307 26378.85
## 20 bet_sign[86]      residence gad7 -0.3112 0.9886 -0.5753 -0.0443 36467.15
## 21 bet_sign[206]   comorbidity gad7 0.2048 0.9937 0.0458 0.3637 38780.84
## 22 bet_sign[101]  consanguinity gad7 -0.1724 0.9783 -0.3393 -0.0045 34300.01
## 23 bet_sign[16]      pr     gad7 -0.1621 0.9939 -0.2905 -0.0341 32639.03
## 24 bet_sign[26]      sb     gad7 0.1109 0.9882 0.0153 0.2070 35337.87
## 25 bet_sign[202]   comorbidity acc -0.3876 0.9996 -0.6051 -0.1671 46080.07
## 26 bet_sign[52]      father   acc 0.3336 0.9892 0.0513 0.6126 39787.89
## 27 bet_sign[187]    c_gender   acc -0.3070 0.9924 -0.5549 -0.0600 51625.04
## 28 bet_sign[157]   time_since acc 0.1500 0.9941 0.0328 0.2672 43102.75
##      Rhat    effect
## 1  1.0000 positive
## 2  1.0001 negative
## 3  1.0000 positive
## 4  1.0001 positive
## 5  1.0000 negative
## 6  0.9999 negative
## 7  0.9999 negative
## 8  1.0000 negative
## 9  0.9999 positive
## 10 1.0000 negative
## 11 1.0001 negative
## 12 1.0000 positive
## 13 1.0001 positive
## 14 1.0000 positive
## 15 1.0000 negative
## 16 1.0000 positive
## 17 1.0001 negative
## 18 1.0000 negative
## 19 1.0002 negative
## 20 1.0000 negative
## 21 1.0001 positive

```

```

## 22 1.0001 negative
## 23 1.0001 negative
## 24 1.0001 positive
## 25 1.0000 negative
## 26 0.9999 positive
## 27 1.0000 negative
## 28 1.0000 positive

## Performing BSEM with Beta prior SD = 0.6

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.005931 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 59.31 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [  0%] (Warmup)
## Chain 1: Iteration:   1300 / 13000 [ 10%] (Warmup)
## Chain 1: Iteration:   2600 / 13000 [ 20%] (Warmup)
## Chain 1: Iteration:   3001 / 13000 [ 23%] (Sampling)
## Chain 1: Iteration:   4300 / 13000 [ 33%] (Sampling)
## Chain 1: Iteration:   5600 / 13000 [ 43%] (Sampling)
## Chain 1: Iteration:   6900 / 13000 [ 53%] (Sampling)
## Chain 1: Iteration:   8200 / 13000 [ 63%] (Sampling)
## Chain 1: Iteration:   9500 / 13000 [ 73%] (Sampling)
## Chain 1: Iteration:  10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration:  12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration:  13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 119.772 seconds (Warm-up)
## Chain 1:                280.949 seconds (Sampling)
## Chain 1:                400.721 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001683 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 16.83 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [  0%] (Warmup)
## Chain 2: Iteration:   1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:   2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:   3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:   4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:   5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:   6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:   8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:   9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration:  10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration:  12100 / 13000 [ 93%] (Sampling)

```

```

## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 93.442 seconds (Warm-up)
## Chain 2: 467.528 seconds (Sampling)
## Chain 2: 560.97 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001596 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 15.96 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 97.012 seconds (Warm-up)
## Chain 3: 500.954 seconds (Sampling)
## Chain 3: 597.966 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001589 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 15.89 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 97.233 seconds (Warm-up)
## Chain 4: 351.254 seconds (Sampling)

```

```

## Chain 4:                      448.487 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>  <chr>
## 1 Divergences    0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI       0.946     > 0.3  No Issue
## 4 Max R-hat      1.00019  < 1.01  No Issue
## 5 Min ESS        33751    > 400   No Issue
##       pxname      from    to    mean    pd    lower    upper    n_eff
## 1  bet_sign[42]    father   sd  0.5703 0.9999 0.2690 0.8695 49623.16
## 2  bet_sign[54]    father   sb -0.4020 0.9973 -0.6859 -0.1174 49081.93
## 3  bet_sign[204]  comorbidity sb 0.3230 0.9982 0.1045 0.5420 54689.06
## 4  bet_sign[189]   c_gender  sb 0.2902 0.9888 0.0425 0.5395 59833.34
## 5  bet_sign[69]    ses       sb -0.2088 0.9997 -0.3294 -0.0880 49841.53
## 6  bet_sign[159]   time_since sb -0.1377 0.9906 -0.2540 -0.0226 51682.06
## 7  bet_sign[199]   comorbidity pr -0.3875 0.9996 -0.6070 -0.1655 55177.68
## 8  bet_sign[184]   c_gender  pr -0.2696 0.9802 -0.5259 -0.0148 54591.27
## 9  bet_sign[64]    ses       pr 0.1908 0.9986 0.0662 0.3158 47570.71
## 10 bet_sign[21]     acc    phq9 -0.2787 0.9997 -0.4250 -0.1307 33751.27
## 11 bet_sign[55]     father phq9 -0.2525 0.9871 -0.4728 -0.0328 38316.04
## 12 bet_sign[25]     sb    phq9 0.2350 1.0000 0.1390 0.3311 46293.36
## 13 bet_sign[145]    parent_cond phq9 0.2270 0.9817 0.0142 0.4372 44934.44
## 14 bet_sign[100]    consanguinity phq9 -0.1816 0.9848 -0.3480 -0.0164 43517.80
## 15 bet_sign[205]    comorbidity phq9 0.1813 0.9863 0.0219 0.3408 47124.76
## 16 bet_sign[160]    time_since phq9 0.1511 0.9997 0.0681 0.2344 44530.30
## 17 bet_sign[1]      sd    phq9 -0.1483 1.0000 -0.2270 -0.0693 46434.55
## 18 bet_sign[15]     pr    phq9 -0.1465 0.9874 -0.2747 -0.0191 39143.69
## 19 bet_sign[196]    comorbidity is -0.2960 0.9960 -0.5164 -0.0735 56972.23
## 20 bet_sign[61]     ses     is 0.2346 1.0000 0.1108 0.3597 48107.90
## 21 bet_sign[166]    delay    is 0.1200 0.9806 0.0067 0.2337 59311.99
## 22 bet_sign[22]     acc    gad7 -0.3801 1.0000 -0.5279 -0.2305 35430.73
## 23 bet_sign[86]     residence gad7 -0.3188 0.9896 -0.5865 -0.0525 47091.41
## 24 bet_sign[206]    comorbidity gad7 0.2072 0.9938 0.0442 0.3673 45976.11
## 25 bet_sign[101]    consanguinity gad7 -0.1753 0.9795 -0.3442 -0.0066 41855.89
## 26 bet_sign[16]     pr    gad7 -0.1616 0.9934 -0.2877 -0.0355 40290.43
## 27 bet_sign[10]     is    gad7 -0.1194 0.9754 -0.2394 -0.0006 37309.40
## 28 bet_sign[26]     sb    gad7 0.1114 0.9878 0.0136 0.2084 45338.91
## 29 bet_sign[202]    comorbidity acc -0.3940 0.9998 -0.6172 -0.1736 60469.09
## 30 bet_sign[52]     father   acc 0.3414 0.9908 0.0562 0.6254 49036.38
## 31 bet_sign[187]    c_gender  acc -0.3105 0.9926 -0.5628 -0.0592 58247.66
## 32 bet_sign[157]    time_since acc 0.1511 0.9938 0.0331 0.2677 54098.98
##       Rhat    effect
## 1  1.0000 positive
## 2  1.0000 negative
## 3  1.0001 positive
## 4  0.9999 positive
## 5  1.0000 negative
## 6  1.0000 negative
## 7  1.0000 negative
## 8  1.0000 negative
## 9  1.0000 positive

```

```

## 10 1.0000 negative
## 11 1.0000 negative
## 12 0.9999 positive
## 13 0.9999 positive
## 14 1.0000 negative
## 15 1.0000 positive
## 16 1.0000 positive
## 17 1.0001 negative
## 18 1.0000 negative
## 19 1.0000 negative
## 20 0.9999 positive
## 21 0.9999 positive
## 22 1.0000 negative
## 23 1.0000 negative
## 24 1.0000 positive
## 25 1.0001 negative
## 26 1.0000 negative
## 27 1.0000 negative
## 28 0.9999 positive
## 29 1.0000 negative
## 30 1.0000 positive
## 31 1.0000 negative
## 32 0.9999 positive

## Performing BSEM with Beta prior SD = 0.7

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001781 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 17.81 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration: 1300 / 13000 [10%] (Warmup)
## Chain 1: Iteration: 2600 / 13000 [20%] (Warmup)
## Chain 1: Iteration: 3001 / 13000 [23%] (Sampling)
## Chain 1: Iteration: 4300 / 13000 [33%] (Sampling)
## Chain 1: Iteration: 5600 / 13000 [43%] (Sampling)
## Chain 1: Iteration: 6900 / 13000 [53%] (Sampling)
## Chain 1: Iteration: 8200 / 13000 [63%] (Sampling)
## Chain 1: Iteration: 9500 / 13000 [73%] (Sampling)
## Chain 1: Iteration: 10800 / 13000 [83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 95.309 seconds (Warm-up)
## Chain 1:           325.221 seconds (Sampling)
## Chain 1:           420.53 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:

```

```

## Chain 2: Gradient evaluation took 0.001642 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 16.42 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 97.342 seconds (Warm-up)
## Chain 2:           392.776 seconds (Sampling)
## Chain 2:           490.118 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001914 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 19.14 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration:  1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration:  2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration:  3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration:  4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration:  5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration:  6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration:  8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration:  9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 95.946 seconds (Warm-up)
## Chain 3:           311.148 seconds (Sampling)
## Chain 3:           407.094 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001942 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 19.42 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:

```

```

## Chain 4:
## Chain 4: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 95.317 seconds (Warm-up)
## Chain 4:           504.444 seconds (Sampling)
## Chain 4:           599.761 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>   <chr>
## 1 Divergences      0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI        0.983    > 0.3  No Issue
## 4 Max R-hat       1.00026  < 1.01 No Issue
## 5 Min ESS         32919    > 400  No Issue
##           pxname      from    to    mean    pd    lower    upper    n_eff
## 1  bet_sign[42]      father    sd  0.5797 0.9998 0.2781 0.8821 48437.07
## 2  bet_sign[54]      father    sb -0.4087 0.9975 -0.6931 -0.1225 46284.57
## 3  bet_sign[204]    comorbidity sb  0.3258 0.9980 0.1084 0.5452 54528.52
## 4  bet_sign[189]      c_gender sb  0.2913 0.9890 0.0418 0.5393 59389.79
## 5  bet_sign[69]       ses      sb -0.2091 0.9993 -0.3319 -0.0870 44618.90
## 6  bet_sign[159]    time_since sb -0.1387 0.9904 -0.2549 -0.0221 50344.55
## 7  bet_sign[199]    comorbidity pr -0.3902 0.9996 -0.6136 -0.1683 59003.30
## 8  bet_sign[184]      c_gender pr -0.2728 0.9837 -0.5223 -0.0209 54605.19
## 9  bet_sign[64]       ses      pr  0.1914 0.9985 0.0673 0.3173 45743.84
## 10 bet_sign[21]      acc    phq9 -0.2797 0.9998 -0.4263 -0.1325 32990.53
## 11 bet_sign[55]      father phq9 -0.2539 0.9878 -0.4753 -0.0319 36031.13
## 12 bet_sign[25]      sb    phq9  0.2344 1.0000 0.1387 0.3320 39778.15
## 13 bet_sign[145]    parent_cond phq9  0.2275 0.9817 0.0142 0.4406 40925.56
## 14 bet_sign[205]    comorbidity phq9  0.1821 0.9882 0.0246 0.3418 44710.76
## 15 bet_sign[100]   consanguinity phq9 -0.1817 0.9837 -0.3514 -0.0139 42418.87
## 16 bet_sign[160]    time_since phq9  0.1512 0.9998 0.0673 0.2352 41104.52
## 17 bet_sign[1]      sd    phq9 -0.1485 1.0000 -0.2276 -0.0699 41741.74
## 18 bet_sign[15]      pr    phq9 -0.1461 0.9862 -0.2749 -0.0158 39360.15
## 19 bet_sign[22]      acc    gad7 -0.3809 1.0000 -0.5297 -0.2320 32919.44
## 20 bet_sign[86]     residence gad7 -0.3227 0.9893 -0.5965 -0.0512 44080.32
## 21 bet_sign[206]    comorbidity gad7  0.2084 0.9946 0.0473 0.3693 45338.26
## 22 bet_sign[101]   consanguinity gad7 -0.1760 0.9797 -0.3451 -0.0082 41498.73
## 23 bet_sign[16]      pr    gad7 -0.1615 0.9926 -0.2908 -0.0319 40348.75
## 24 bet_sign[26]      sb    gad7  0.1110 0.9866 0.0130 0.2091 40051.76
## 25 bet_sign[202]    comorbidity acc -0.3977 0.9998 -0.6183 -0.1771 56461.81
## 26 bet_sign[52]     father    acc  0.3455 0.9898 0.0569 0.6345 47841.66

```

```

## 27 bet_sign[187]      c_gender  acc -0.3132 0.9927 -0.5634 -0.0635 57174.26
## 28 bet_sign[157]      time_since acc  0.1524 0.9940  0.0352  0.2698 51530.22
##      Rhat    effect
## 1  1.0000 positive
## 2  1.0000 negative
## 3  0.9999 positive
## 4  0.9999 positive
## 5  0.9999 negative
## 6  1.0000 negative
## 7  0.9999 negative
## 8  1.0000 negative
## 9  1.0000 positive
## 10 1.0000 negative
## 11 1.0001 negative
## 12 1.0000 positive
## 13 1.0001 positive
## 14 0.9999 positive
## 15 1.0000 negative
## 16 1.0000 positive
## 17 1.0000 negative
## 18 1.0000 negative
## 19 1.0000 negative
## 20 1.0000 negative
## 21 1.0000 positive
## 22 0.9999 negative
## 23 1.0000 negative
## 24 1.0001 positive
## 25 1.0000 negative
## 26 0.9999 positive
## 27 1.0000 negative
## 28 1.0000 positive

```

```
## Performing BSEM with Beta prior SD = 0.8
```

```
##
```

```
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
```

```
## Chain 1:
```

```
## Chain 1: Gradient evaluation took 0.002043 seconds
```

```
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 20.43 seconds.
```

```
## Chain 1: Adjust your expectations accordingly!
```

```
## Chain 1:
```

```
## Chain 1:
```

```

## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration:  1300 / 13000 [10%] (Warmup)
## Chain 1: Iteration:  2600 / 13000 [20%] (Warmup)
## Chain 1: Iteration:  3001 / 13000 [23%] (Sampling)
## Chain 1: Iteration:  4300 / 13000 [33%] (Sampling)
## Chain 1: Iteration:  5600 / 13000 [43%] (Sampling)
## Chain 1: Iteration:  6900 / 13000 [53%] (Sampling)
## Chain 1: Iteration:  8200 / 13000 [63%] (Sampling)
## Chain 1: Iteration:  9500 / 13000 [73%] (Sampling)
## Chain 1: Iteration: 10800 / 13000 [83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)

```

```

## Chain 1:
## Chain 1: Elapsed Time: 95.226 seconds (Warm-up)
## Chain 1: 271.122 seconds (Sampling)
## Chain 1: 366.348 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001681 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 16.81 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 96.364 seconds (Warm-up)
## Chain 2: 374.575 seconds (Sampling)
## Chain 2: 470.939 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001677 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 16.77 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 96.592 seconds (Warm-up)
## Chain 3: 393.841 seconds (Sampling)
## Chain 3: 490.433 seconds (Total)

```

```

## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001598 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 15.98 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration:   1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration:   2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration:   3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration:   4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration:   5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration:   6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration:   8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration:   9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration:  10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration:  12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration:  13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 96.089 seconds (Warm-up)
## Chain 4:                496.853 seconds (Sampling)
## Chain 4:                592.942 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>  <chr>
## 1 Divergences      0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI        0.972    > 0.3    No Issue
## 4 Max R-hat       1.00016 < 1.01 No Issue
## 5 Min ESS        33033    > 400    No Issue
##           pxname      from    to    mean    pd    lower    upper    n_eff
## 1  bet_sign[42]      father    sd  0.5886 0.9998 0.2846 0.8897 47658.30
## 2  bet_sign[54]      father    sb -0.4113 0.9974 -0.6990 -0.1242 46753.68
## 3  bet_sign[204]    comorbidity sb  0.3268 0.9976 0.1048 0.5480 54962.32
## 4  bet_sign[189]      c_gender sb  0.2940 0.9883 0.0408 0.5477 57401.68
## 5  bet_sign[69]       ses      sb -0.2092 0.9995 -0.3325 -0.0868 45211.23
## 6  bet_sign[159]    time_since sb -0.1394 0.9912 -0.2543 -0.0242 48291.49
## 7  bet_sign[199]    comorbidity pr -0.3928 0.9998 -0.6138 -0.1691 54317.27
## 8  bet_sign[184]      c_gender pr -0.2744 0.9824 -0.5288 -0.0197 54234.52
## 9  bet_sign[64]       ses      pr  0.1911 0.9985 0.0649 0.3163 47875.76
## 10 bet_sign[21]      acc phq9 -0.2816 0.9998 -0.4292 -0.1329 34381.09
## 11 bet_sign[55]      father phq9 -0.2549 0.9878 -0.4750 -0.0345 37372.57
## 12 bet_sign[25]      sb phq9  0.2349 1.0000 0.1383 0.3312 46194.75
## 13 bet_sign[145]    parent_cond phq9  0.2281 0.9825 0.0168 0.4401 44503.74
## 14 bet_sign[100]  consanguinity phq9 -0.1841 0.9838 -0.3540 -0.0163 43888.71
## 15 bet_sign[205]    comorbidity phq9  0.1836 0.9878 0.0249 0.3436 46679.72
## 16 bet_sign[160]    time_since phq9  0.1515 0.9999 0.0681 0.2354 39616.12
## 17 bet_sign[1]      sd phq9 -0.1489 0.9999 -0.2280 -0.0699 43611.05
## 18 bet_sign[15]      pr phq9 -0.1452 0.9855 -0.2742 -0.0146 39752.53

```

```

## 19 bet_sign[22]          acc gad7 -0.3823 1.0000 -0.5321 -0.2342 33135.50
## 20 bet_sign[86]         residence gad7 -0.3244 0.9898 -0.5978 -0.0491 43531.72
## 21 bet_sign[206]        comorbidity gad7 0.2100 0.9951 0.0481 0.3720 46991.87
## 22 bet_sign[101] consanguinity gad7 -0.1783 0.9812 -0.3470 -0.0101 44149.48
## 23 bet_sign[16]         pr gad7 -0.1610 0.9924 -0.2893 -0.0323 38672.40
## 24 bet_sign[26]         sb gad7 0.1107 0.9867 0.0137 0.2081 44552.46
## 25 bet_sign[202]        comorbidity acc -0.4004 0.9998 -0.6227 -0.1780 56201.96
## 26 bet_sign[52]         father acc 0.3482 0.9893 0.0535 0.6418 49391.81
## 27 bet_sign[187]        c_gender acc -0.3150 0.9926 -0.5667 -0.0630 52615.57
## 28 bet_sign[157]        time_since acc 0.1522 0.9949 0.0347 0.2707 49573.67
##      Rhat      effect
## 1 0.9999 positive
## 2 1.0000 negative
## 3 1.0000 positive
## 4 1.0000 positive
## 5 1.0000 negative
## 6 0.9999 negative
## 7 1.0000 negative
## 8 0.9999 negative
## 9 0.9999 positive
## 10 1.0000 negative
## 11 1.0000 negative
## 12 1.0000 positive
## 13 0.9999 positive
## 14 1.0000 negative
## 15 1.0000 positive
## 16 1.0000 positive
## 17 1.0000 negative
## 18 1.0001 negative
## 19 1.0000 negative
## 20 1.0000 negative
## 21 1.0000 positive
## 22 1.0000 negative
## 23 1.0000 negative
## 24 1.0001 positive
## 25 1.0000 negative
## 26 1.0000 positive
## 27 1.0000 negative
## 28 0.9999 positive

```

```
## Performing BSEM with Beta prior SD = 0.9
```

```

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001776 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 17.76 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration: 1300 / 13000 [10%] (Warmup)
## Chain 1: Iteration: 2600 / 13000 [20%] (Warmup)
## Chain 1: Iteration: 3001 / 13000 [23%] (Sampling)

```

```

## Chain 1: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 1: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 1: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 1: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 1: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 1: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 95.538 seconds (Warm-up)
## Chain 1: 473.115 seconds (Sampling)
## Chain 1: 568.653 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001826 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 18.26 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 97.505 seconds (Warm-up)
## Chain 2: 257.073 seconds (Sampling)
## Chain 2: 354.578 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001588 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 15.88 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:
## Chain 3: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration: 8200 / 13000 [ 63%] (Sampling)

```

```

## Chain 3: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 96.425 seconds (Warm-up)
## Chain 3: 391 seconds (Sampling)
## Chain 3: 487.425 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001865 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 18.65 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration: 1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration: 1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration: 2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration: 3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration: 4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration: 5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration: 6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration: 8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration: 9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration: 10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration: 12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration: 13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 97.919 seconds (Warm-up)
## Chain 4: 396.317 seconds (Sampling)
## Chain 4: 494.236 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
## Diagnostic Value Cutoff Convergence
## <chr> <chr> <chr> <chr>
## 1 Divergences 0 = 0 No Issue
## 2 Treedepth Hits 0.000% < 0.1% No Issue
## 3 Min BFMI 0.976 > 0.3 No Issue
## 4 Max R-hat 1.00013 < 1.01 No Issue
## 5 Min ESS 34072 > 400 No Issue
## pxname from to mean pd lower upper n_eff
## 1 bet_sign[42] father sd 0.5946 0.9999 0.2912 0.8963 46926.97
## 2 bet_sign[54] father sb -0.4141 0.9971 -0.7024 -0.1218 48912.29
## 3 bet_sign[204] comorbidity sb 0.3283 0.9980 0.1058 0.5469 56018.45
## 4 bet_sign[189] c_gender sb 0.2956 0.9897 0.0479 0.5465 57017.60
## 5 bet_sign[69] ses sb -0.2097 0.9995 -0.3331 -0.0878 42678.05
## 6 bet_sign[159] time_since sb -0.1402 0.9910 -0.2556 -0.0245 52485.77
## 7 bet_sign[199] comorbidity pr -0.3943 0.9998 -0.6196 -0.1696 54591.70
## 8 bet_sign[184] c_gender pr -0.2750 0.9811 -0.5329 -0.0136 57098.67
## 9 bet_sign[64] ses pr 0.1913 0.9986 0.0657 0.3164 47070.87
## 10 bet_sign[21] acc phq9 -0.2813 0.9999 -0.4272 -0.1332 34072.46

```

```

## 11 bet_sign[55]      father phq9 -0.2561 0.9885 -0.4757 -0.0349 38009.22
## 12 bet_sign[25]      sb phq9  0.2350 1.0000  0.1384  0.3323 42325.99
## 13 bet_sign[145]     parent_cond phq9  0.2296 0.9826  0.0164  0.4443 45271.92
## 14 bet_sign[205]     comorbidity phq9  0.1840 0.9873  0.0240  0.3437 46200.09
## 15 bet_sign[100]     consanguinity phq9 -0.1837 0.9838 -0.3517 -0.0161 43913.25
## 16 bet_sign[160]     time_since phq9  0.1515 0.9998  0.0682  0.2346 45119.33
## 17 bet_sign[1]       sd phq9 -0.1481 0.9998 -0.2271 -0.0692 43927.78
## 18 bet_sign[15]      pr phq9 -0.1460 0.9875 -0.2737 -0.0182 42258.07
## 19 bet_sign[22]      acc gad7 -0.3825 1.0000 -0.5311 -0.2326 34646.88
## 20 bet_sign[86]      residence gad7 -0.3252 0.9906 -0.5972 -0.0534 42476.11
## 21 bet_sign[206]     comorbidity gad7  0.2099 0.9938  0.0476  0.3722 48107.99
## 22 bet_sign[101]     consanguinity gad7 -0.1782 0.9806 -0.3454 -0.0080 42449.75
## 23 bet_sign[16]      pr gad7 -0.1613 0.9926 -0.2896 -0.0325 41309.19
## 24 bet_sign[26]      sb gad7  0.1108 0.9873  0.0141  0.2082 42626.67
## 25 bet_sign[202]     comorbidity acc -0.4012 0.9998 -0.6209 -0.1817 55712.22
## 26 bet_sign[52]      father acc  0.3503 0.9913  0.0611  0.6405 50932.67
## 27 bet_sign[187]     c_gender acc -0.3167 0.9933 -0.5691 -0.0636 56055.04
## 28 bet_sign[157]     time_since acc  0.1528 0.9949  0.0362  0.2702 52923.65
##      Rhat      effect
## 1  0.9999 positive
## 2  0.9999 negative
## 3  1.0000 positive
## 4  0.9999 positive
## 5  1.0001 negative
## 6  0.9999 negative
## 7  0.9999 negative
## 8  1.0000 negative
## 9  1.0000 positive
## 10 0.9999 negative
## 11 1.0000 negative
## 12 1.0000 positive
## 13 1.0001 positive
## 14 1.0000 positive
## 15 1.0000 negative
## 16 0.9999 positive
## 17 1.0001 negative
## 18 0.9999 negative
## 19 1.0000 negative
## 20 1.0001 negative
## 21 1.0000 positive
## 22 1.0000 negative
## 23 1.0000 negative
## 24 1.0000 positive
## 25 1.0000 negative
## 26 0.9999 positive
## 27 1.0000 negative
## 28 1.0000 positive

```

```
## Performing BSEM with Beta prior SD = 1
```

```

##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.001745 seconds

```

```

## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 17.45 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 1: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 1: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 1: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 1: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 1: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 1: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 1: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 1: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 1: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 1: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 1: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 1:
## Chain 1: Elapsed Time: 97.551 seconds (Warm-up)
## Chain 1:           257.128 seconds (Sampling)
## Chain 1:           354.679 seconds (Total)
## Chain 1:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 2).
## Chain 2:
## Chain 2: Gradient evaluation took 0.001758 seconds
## Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 17.58 seconds.
## Chain 2: Adjust your expectations accordingly!
## Chain 2:
## Chain 2:
## Chain 2: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 2: Iteration:    1300 / 13000 [ 10%] (Warmup)
## Chain 2: Iteration:    2600 / 13000 [ 20%] (Warmup)
## Chain 2: Iteration:    3001 / 13000 [ 23%] (Sampling)
## Chain 2: Iteration:    4300 / 13000 [ 33%] (Sampling)
## Chain 2: Iteration:    5600 / 13000 [ 43%] (Sampling)
## Chain 2: Iteration:    6900 / 13000 [ 53%] (Sampling)
## Chain 2: Iteration:    8200 / 13000 [ 63%] (Sampling)
## Chain 2: Iteration:    9500 / 13000 [ 73%] (Sampling)
## Chain 2: Iteration:   10800 / 13000 [ 83%] (Sampling)
## Chain 2: Iteration:   12100 / 13000 [ 93%] (Sampling)
## Chain 2: Iteration:   13000 / 13000 [100%] (Sampling)
## Chain 2:
## Chain 2: Elapsed Time: 96.356 seconds (Warm-up)
## Chain 2:           264.2 seconds (Sampling)
## Chain 2:           360.556 seconds (Total)
## Chain 2:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 3).
## Chain 3:
## Chain 3: Gradient evaluation took 0.001659 seconds
## Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 16.59 seconds.
## Chain 3: Adjust your expectations accordingly!
## Chain 3:
## Chain 3:

```

```

## Chain 3: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 3: Iteration:   1300 / 13000 [ 10%] (Warmup)
## Chain 3: Iteration:   2600 / 13000 [ 20%] (Warmup)
## Chain 3: Iteration:   3001 / 13000 [ 23%] (Sampling)
## Chain 3: Iteration:   4300 / 13000 [ 33%] (Sampling)
## Chain 3: Iteration:   5600 / 13000 [ 43%] (Sampling)
## Chain 3: Iteration:   6900 / 13000 [ 53%] (Sampling)
## Chain 3: Iteration:   8200 / 13000 [ 63%] (Sampling)
## Chain 3: Iteration:   9500 / 13000 [ 73%] (Sampling)
## Chain 3: Iteration:  10800 / 13000 [ 83%] (Sampling)
## Chain 3: Iteration:  12100 / 13000 [ 93%] (Sampling)
## Chain 3: Iteration:  13000 / 13000 [100%] (Sampling)
## Chain 3:
## Chain 3: Elapsed Time: 96.741 seconds (Warm-up)
## Chain 3:           503.961 seconds (Sampling)
## Chain 3:           600.702 seconds (Total)
## Chain 3:
##
## SAMPLING FOR MODEL 'stanmarg' NOW (CHAIN 4).
## Chain 4:
## Chain 4: Gradient evaluation took 0.001599 seconds
## Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 15.99 seconds.
## Chain 4: Adjust your expectations accordingly!
## Chain 4:
## Chain 4:
## Chain 4: Iteration:      1 / 13000 [ 0%] (Warmup)
## Chain 4: Iteration:   1300 / 13000 [ 10%] (Warmup)
## Chain 4: Iteration:   2600 / 13000 [ 20%] (Warmup)
## Chain 4: Iteration:   3001 / 13000 [ 23%] (Sampling)
## Chain 4: Iteration:   4300 / 13000 [ 33%] (Sampling)
## Chain 4: Iteration:   5600 / 13000 [ 43%] (Sampling)
## Chain 4: Iteration:   6900 / 13000 [ 53%] (Sampling)
## Chain 4: Iteration:   8200 / 13000 [ 63%] (Sampling)
## Chain 4: Iteration:   9500 / 13000 [ 73%] (Sampling)
## Chain 4: Iteration:  10800 / 13000 [ 83%] (Sampling)
## Chain 4: Iteration:  12100 / 13000 [ 93%] (Sampling)
## Chain 4: Iteration:  13000 / 13000 [100%] (Sampling)
## Chain 4:
## Chain 4: Elapsed Time: 95.059 seconds (Warm-up)
## Chain 4:           269.575 seconds (Sampling)
## Chain 4:           364.634 seconds (Total)
## Chain 4:
## Computing post-estimation metrics (including lvs if requested)...
## # A tibble: 5 x 4
##   Diagnostic      Value      Cutoff Convergence
##   <chr>          <chr>    <chr>  <chr>
## 1 Divergences      0        = 0    No Issue
## 2 Treedepth Hits 0.000%    < 0.1% No Issue
## 3 Min BFMI        0.966    > 0.3  No Issue
## 4 Max R-hat       1.00020 < 1.01 No Issue
## 5 Min ESS         30486    > 400  No Issue
##           pxname           from to mean   pd  lower  upper  n_eff
## 1  bet_sign[42]      father  sd  0.5986 1.0000 0.2959 0.9014 46799.22
## 2  bet_sign[54]      father  sb -0.4169 0.9979 -0.7082 -0.1262 46289.71

```

```

## 3 bet_sign[204] comorbidity sb 0.3294 0.9986 0.1101 0.5508 59755.38
## 4 bet_sign[189] c_gender sb 0.2965 0.9888 0.0442 0.5498 59018.60
## 5 bet_sign[69] ses sb -0.2096 0.9996 -0.3330 -0.0859 44529.43
## 6 bet_sign[159] time_since sb -0.1400 0.9912 -0.2552 -0.0244 50239.18
## 7 bet_sign[199] comorbidity pr -0.3961 0.9997 -0.6200 -0.1735 54403.97
## 8 bet_sign[184] c_gender pr -0.2767 0.9824 -0.5352 -0.0188 64054.28
## 9 bet_sign[64] ses pr 0.1915 0.9987 0.0664 0.3168 45966.07
## 10 bet_sign[21] acc phq9 -0.2817 0.9999 -0.4300 -0.1346 30929.88
## 11 bet_sign[55] father phq9 -0.2568 0.9883 -0.4801 -0.0358 36170.72
## 12 bet_sign[25] sb phq9 0.2349 1.0000 0.1375 0.3320 39979.84
## 13 bet_sign[145] parent_cond phq9 0.2309 0.9824 0.0162 0.4438 44542.12
## 14 bet_sign[100] consanguinity phq9 -0.1856 0.9846 -0.3536 -0.0177 39904.70
## 15 bet_sign[205] comorbidity phq9 0.1846 0.9880 0.0251 0.3453 47140.50
## 16 bet_sign[160] time_since phq9 0.1514 0.9999 0.0687 0.2348 41474.72
## 17 bet_sign[1] sd phq9 -0.1481 0.9999 -0.2274 -0.0684 41192.43
## 18 bet_sign[15] pr phq9 -0.1456 0.9865 -0.2750 -0.0175 38023.03
## 19 bet_sign[22] acc gad7 -0.3830 1.0000 -0.5328 -0.2329 30486.25
## 20 bet_sign[86] residence gad7 -0.3283 0.9913 -0.6000 -0.0575 43963.54
## 21 bet_sign[206] comorbidity gad7 0.2106 0.9948 0.0486 0.3726 44454.52
## 22 bet_sign[101] consanguinity gad7 -0.1794 0.9814 -0.3482 -0.0106 41029.42
## 23 bet_sign[16] pr gad7 -0.1610 0.9924 -0.2899 -0.0314 37640.54
## 24 bet_sign[26] sb gad7 0.1107 0.9861 0.0129 0.2080 40462.39
## 25 bet_sign[202] comorbidity acc -0.4027 0.9998 -0.6271 -0.1783 62294.80
## 26 bet_sign[52] father acc 0.3515 0.9897 0.0568 0.6484 52108.60
## 27 bet_sign[187] c_gender acc -0.3177 0.9922 -0.5741 -0.0650 65956.74
## 28 bet_sign[157] time_since acc 0.1530 0.9942 0.0346 0.2703 56514.04
## Rhat effect
## 1 1.0000 positive
## 2 1.0000 negative
## 3 1.0000 positive
## 4 1.0000 positive
## 5 1.0000 negative
## 6 1.0000 negative
## 7 0.9999 negative
## 8 0.9999 negative
## 9 1.0000 positive
## 10 1.0000 negative
## 11 1.0000 negative
## 12 1.0000 positive
## 13 1.0000 positive
## 14 1.0000 negative
## 15 1.0000 positive
## 16 1.0000 positive
## 17 0.9999 negative
## 18 1.0000 negative
## 19 1.0000 negative
## 20 1.0000 negative
## 21 1.0001 positive
## 22 1.0000 negative
## 23 1.0000 negative
## 24 1.0000 positive
## 25 1.0001 negative
## 26 1.0000 positive
## 27 0.9999 negative

```

```
## 28 0.9999 positive
## # A tibble: 25 x 9
##   from      to      k      n      H      df p_value eta_sq effect_label
##   <chr>    <chr> <int> <int> <dbl> <dbl> <dbl> <dbl> <chr>
## 1 father    sd      9 360000 67413.      8      0 0.187 Large
## 2 father    sb      9 360000 30152.      8      0 0.0837 Moderate
## 3 comorbidity acc      9 360000 22110.      8      0 0.0614 Moderate
## 4 comorbidity pr       9 360000 20409.      8      0 0.0567 Small
## 5 father    acc      9 360000 19645.      8      0 0.0545 Small
## 6 residence gad7     9 360000 17486.      8      0 0.0486 Small
## 7 comorbidity sb      9 360000 13086.      8      0 0.0363 Small
## 8 c_gender  acc      9 360000 11381.      8      0 0.0316 Small
## 9 c_gender  sb      9 360000 10527.      8      0 0.0292 Small
## 10 c_gender pr       9 360000 8806.       8      0 0.0244 Small
## # i 15 more rows
```

